



# TEAM TASK BREAKDOWN - 4 PEOPLE

## (COLLABORATIVE)

**Competition:** CSIRO Image2Biomass

**Deadline:** January 28, 2026 (59 days remaining)

**Target:** 0.82-0.85  $R^2$  (Top 5-10%)

**Philosophy:** Everyone works on every phase. Breadth over depth.

### PHASE 1: DATA PREPARATION

**Timeline:** Days 1-2 (2 days)

**All 4 people contribute**

#### PERSON 1: Data Download & EDA

**Time:** 3 hours

```
# Task 1A: Download & organize
kaggle competitions download -c csiro-image2biomass
# Structure: data/train.csv, data/train_images/, data/test.csv, data/test_images/

# Task 1B: Basic EDA
import pandas as pd
import numpy as np

train_df = pd.read_csv('data/train.csv')

print(f"Shape: {train_df.shape}")
print(f"Missing values:\n{train_df.isnull().sum()}")
print(f"\nTarget ranges:")
print(train_df[['Dry_Green_g', 'Dry_Dead_g', 'Dry_Clover_g', 'GDM_g', 'Dry_Total_g']].des

# Correlations
for target in ['Dry_Green_g', 'GDM_g', 'Dry_Total_g']:
    ndvi_corr = np.corrcoef(train_df['Pre_GSHH_NDVI'], train_df[target])[0,1]
    height_corr = np.corrcoef(train_df['Height_Ave_cm'], train_df[target])[0,1]
    print(f"{target}: NDVI r={ndvi_corr:.3f}, Height r={height_corr:.3f}")

# Save findings
with open('eda_report.txt', 'w') as f:
    f.write(f"Training samples: {len(train_df)}\n")
```

```
f.write(f"States: {train_df['State'].nunique()}\n")
f.write(f"Species: {train_df['Species'].nunique()}\n")
```

**Deliverable:** eda\_report.txt, data organized in data/ folder

## PERSON 2: Tabular Feature Engineering

**Time:** 3 hours

```
# Task 2A: Feature engineering pipeline
import pandas as pd
import numpy as np
import pickle

class TabularPreprocessor:
    def __init__(self):
        self.ndvi_min = 0.28
        self.ndvi_max = 0.89
        self.height_mean = None
        self.height_std = None
        self.state_map = {}
        self.species_map = {}

    def fit(self, df):
        # Height statistics
        height_log = np.log(df['Height_Ave_cm'].values + 0.1)
        self.height_mean = height_log.mean()
        self.height_std = height_log.std()

        # Categorical mappings
        self.state_map = {s: i for i, s in enumerate(sorted(df['State'].unique()))}
        self.species_map = {s: i for i, s in enumerate(sorted(df['Species'].unique()))}
        return self

    def transform(self, df):
        processed = df.copy()

        # NDVI normalization
        processed['ndvi_norm'] = (df['Pre_GSHH_NDVI'] - self.ndvi_min) / (self.ndvi_max - self.ndvi_min)

        # Height log-transform
        height_log = np.log(df['Height_Ave_cm'].values + 0.1)
        processed['height_norm'] = (height_log - self.height_mean) / (self.height_std + 1e-6)

        # Date cyclical encoding
        dates = pd.to_datetime(df['Sampling_Date'])
        doy = dates.dt.dayofyear.values
        processed['date_sin'] = np.sin(2 * np.pi * doy / 365)
        processed['date_cos'] = np.cos(2 * np.pi * doy / 365)
        processed['date_sin2'] = np.sin(4 * np.pi * doy / 365)
        processed['date_cos2'] = np.cos(4 * np.pi * doy / 365)

        # Categorical
        processed['state_id'] = df['State'].map(self.state_map)
```

```

        processed['species_id'] = df['Species'].map(self.species_map)

    return processed

# Apply
train_df = pd.read_csv('data/train.csv')
preprocessor = TabularPreprocessor().fit(train_df)
train_processed = preprocessor.transform(train_df)
train_processed.to_csv('data/train_processed.csv', index=False)

# Save preprocessor
with open('data/preprocessor.pkl', 'wb') as f:
    pickle.dump(preprocessor, f)

print(f"✓ Processed {len(train_processed)} samples")
print(f"  States: {preprocessor.state_map}")
print(f"  Species count: {len(preprocessor.species_map)}")

```

**Deliverable:** data/train\_processed.csv, data/preprocessor.pkl

## PERSON 3: Dataset Class

**Time:** 3 hours

```

# Task 3A: BiomassDataset implementation
import torch
from torch.utils.data import Dataset
import pandas as pd
import numpy as np
from PIL import Image

class BiomassDataset(Dataset):
    def __init__(self, df, img_dir, transform=None, mode='train'):
        self.df = df.reset_index(drop=True)
        self.img_dir = img_dir
        self.transform = transform
        self.mode = mode
        self.targets = ['Dry_Green_g', 'Dry_Dead_g', 'Dry_Clover_g', 'GDM_g', 'Dry_Total_

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        # Load image
        img_id = self.df.loc[idx, 'sample_id']
        img_path = f"{self.img_dir}/{img_id}.jpg"
        image = np.array(Image.open(img_path).convert('RGB'))

        # Apply transforms
        if self.transform:
            image = self.transform(image=image)['image']

        # Tabular features
        tabular = {

```

```

        'ndvi': torch.tensor(self.df.loc[idx, 'ndvi_norm'], dtype=torch.float32),
        'height': torch.tensor(self.df.loc[idx, 'height_norm'], dtype=torch.float32),
        'date_sin': torch.tensor(self.df.loc[idx, 'date_sin'], dtype=torch.float32),
        'date_cos': torch.tensor(self.df.loc[idx, 'date_cos'], dtype=torch.float32),
        'date_sin2': torch.tensor(self.df.loc[idx, 'date_sin2'], dtype=torch.float32),
        'date_cos2': torch.tensor(self.df.loc[idx, 'date_cos2'], dtype=torch.float32),
        'state_id': torch.tensor(self.df.loc[idx, 'state_id'], dtype=torch.long),
        'species_id': torch.tensor(self.df.loc[idx, 'species_id'], dtype=torch.long),
    }

    if self.mode == 'train':
        targets = {t: torch.tensor(self.df.loc[idx, t], dtype=torch.float32)
                    for t in self.targets}
        return image, tabular, targets
    else:
        return image, tabular

# Test
if __name__ == '__main__':
    from torch.utils.data import DataLoader
    import albumentations as A
    from albumentations.pytorch import ToTensorV2

    transform = A.Compose([
        A.Resize(518, 518),
        A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
        ToTensorV2()
    ])

    df = pd.read_csv('data/train_processed.csv')
    dataset = BiomassDataset(df, 'data/train_images', transform)
    loader = DataLoader(dataset, batch_size=4, shuffle=True)

    images, tabular, targets = next(iter(loader))
    print(f"✓ Dataset test passed")
    print(f"  Images: {images.shape}")
    print(f"  Targets: {list(targets.keys())}")

```

**Deliverable:** dataset.py (tested and working)

## PERSON 4: Augmentation Transforms

**Time:** 2 hours

```

# Task 4A: All transform pipelines
import albumentations as A
from albumentations.pytorch import ToTensorV2

def get_train_transforms(img_size=518):
    """Training: spatial + color augmentations"""
    return A.Compose([
        A.Resize(img_size, img_size),
        A.HorizontalFlip(p=0.5),
        A.VerticalFlip(p=0.5),

```

```

        A.Rotate(limit=15, p=0.5),
        A.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, p=0.5),
        A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
        ToTensorV2()
    ])

def get_val_transforms(img_size=518):
    """Validation: resize + normalize only"""
    return A.Compose([
        A.Resize(img_size, img_size),
        A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
        ToTensorV2()
    ])

def get_tta_transforms(img_size=518):
    """8 deterministic views for TTA"""
    base = [A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]), ToTensorV2()]
    return [
        A.Compose([A.Resize(img_size, img_size)] + base), # 1. Original
        A.Compose([A.Resize(img_size, img_size), A.HorizontalFlip(p=1.0)] + base), # 2.
        A.Compose([A.Resize(img_size, img_size), A.VerticalFlip(p=1.0)] + base), # 3. V-
        A.Compose([A.Resize(img_size, img_size), A.Rotate(limit=(10,10), p=1.0)] + base),
        A.Compose([A.Resize(img_size, img_size), A.Rotate(limit=(-10,-10), p=1.0)] + base),
        A.Compose([A.Resize(img_size, img_size), A.ColorJitter(brightness=0.1, p=1.0)] +
        A.Compose([A.Resize(img_size, img_size), A.HorizontalFlip(p=1.0), A.Rotate(limit=
        A.Compose([A.Resize(img_size, img_size), A.VerticalFlip(p=1.0), A.Rotate(limit=
    ]

# Visualize augmentations
if __name__ == '__main__':
    import matplotlib.pyplot as plt
    from PIL import Image
    import numpy as np

    img = np.array(Image.open('data/train_images/ID0001.jpg'))
    transforms = get_tta_transforms()

    fig, axes = plt.subplots(2, 4, figsize=(16, 8))
    for i, (ax, transform) in enumerate(zip(axes.flat, transforms)):
        aug = transform(image=img)['image']
        aug = aug.permute(1, 2, 0).numpy()
        aug = (aug * [0.229, 0.224, 0.225]) + [0.485, 0.456, 0.406]
        aug = np.clip(aug, 0, 1)
        ax.imshow(aug)
        ax.set_title(f"TTA View {i+1}")
        ax.axis('off')
    plt.tight_layout()
    plt.savefig('tta_visualization.png')
    print("✓ TTA visualization saved")

```

**Deliverable:** transforms.py, tta\_visualization.png

## PHASE 2: MODEL ARCHITECTURE

**Timeline:** Days 3-4 (2 days)

**All 4 people contribute**

### PERSON 1: TabularEmbedder Module

**Time:** 3 hours

```
# Task 1B: Tabular feature embedder
import torch
import torch.nn as nn

class TabularEmbedder(nn.Module):
    """Embeds tabular features to 768-dim (DINO dimension)"""
    def __init__(self, num_states, num_species, output_dim=768):
        super().__init__()

        # Per-feature embedders
        self.ndvi_embed = nn.Sequential(
            nn.Linear(1, 32), nn.LayerNorm(32), nn.ReLU(), nn.Linear(32, 64)
        )
        self.height_embed = nn.Sequential(
            nn.Linear(1, 32), nn.LayerNorm(32), nn.ReLU(), nn.Linear(32, 64)
        )
        self.date_embed = nn.Sequential(
            nn.Linear(4, 32), nn.LayerNorm(32), nn.ReLU(), nn.Linear(32, 64)
        )

        # Categorical embeddings
        self.state_embedding = nn.Embedding(num_states, 8)
        self.species_embedding = nn.Embedding(num_species, 16)

        # Fusion MLP: 64+64+64+8+16+1 = 217 → 768
        self.fusion = nn.Sequential(
            nn.Linear(217, 128), nn.LayerNorm(128), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(128, 256), nn.LayerNorm(256), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(256, output_dim)
        )

    def forward(self, tabular_dict):
        # Embed continuous features
        ndvi_emb = self.ndvi_embed(tabular_dict['ndvi'].unsqueeze(-1))
        height_emb = self.height_embed(tabular_dict['height'].unsqueeze(-1))

        date_feats = torch.stack([
            tabular_dict['date_sin'], tabular_dict['date_cos'],
            tabular_dict['date_sin2'], tabular_dict['date_cos2']
        ], dim=-1)
        date_emb = self.date_embed(date_feats)

        # Embed categorical
        state_emb = self.state_embedding(tabular_dict['state_id'])
        species_emb = self.species_embedding(tabular_dict['species_id'])
```

```

        # Interaction term
        interaction = (tabular_dict['ndvi'] * tabular_dict['height']).unsqueeze(-1) * 100

        # Concatenate & fuse
        combined = torch.cat([ndvi_emb, height_emb, date_emb, state_emb, species_emb, interaction_emb])
        return self.fusion(combined)

# Unit test
if __name__ == '__main__':
    embedder = TabularEmbedder(num_states=20, num_species=35)
    dummy = {
        'ndvi': torch.randn(4), 'height': torch.randn(4),
        'date_sin': torch.randn(4), 'date_cos': torch.randn(4),
        'date_sin2': torch.randn(4), 'date_cos2': torch.randn(4),
        'state_id': torch.randint(0, 20, (4,)), 'species_id': torch.randint(0, 35, (4,))
    }
    output = embedder(dummy)
    assert output.shape == (4, 768)
    print("✓ TabularEmbedder test passed")

```

**Deliverable:** TabularEmbedder class in model.py

## PERSON 2: DINO Backbone Wrapper

**Time:** 2 hours

```

# Task 2B: DINO v2 backbone
import torch
import torch.nn as nn
import timm

class DINOBackbone(nn.Module):
    """DINO v2-B/14 with selective fine-tuning"""
    def __init__(self, freeze_backbone_pct=0.7):
        super().__init__()

        # Load pretrained DINO v2
        self.dino = timm.create_model(
            'vit_base_patch14_dinov2.lvd142m',
            pretrained=True,
            num_classes=0 # Remove classification head
        )

        # Freeze early layers (general features)
        total_blocks = len(list(self.dino.blocks))
        freeze_until = int(total_blocks * freeze_backbone_pct)

        for i, block in enumerate(self.dino.blocks):
            if i < freeze_until:
                for param in block.parameters():
                    param.requires_grad = False

        print(f"DINO: Frozen {freeze_until}/{total_blocks} blocks")

```

```

def forward(self, x):
    """
    Args: x (B, 3, 518, 518)
    Returns: patch_tokens (B, 196, 768)
    """
    features = self.dino.forward_features(x) # (B, 197, 768) with CLS
    return features[:, 1:, :] # Remove CLS token

# Unit test
if __name__ == '__main__':
    backbone = DINOBackbone()
    dummy_img = torch.randn(2, 3, 518, 518)
    output = backbone(dummy_img)
    assert output.shape == (2, 196, 768)

    # Check frozen parameters
    trainable = sum(p.numel() for p in backbone.parameters() if p.requires_grad)
    total = sum(p.numel() for p in backbone.parameters())
    print(f"✓ DINO test passed")
    print(f"  Trainable: {trainable:,} / {total:,} ({100*trainable/total:.1f}%)")

```

**Deliverable:** DINOBackbone class in model.py

## PERSON 3: Cross-Modal Fusion Layer

**Time:** 3 hours

```

# Task 3B: Cross-attention fusion
import torch
import torch.nn as nn

class CrossModalFusion(nn.Module):
    """Fuses image patches with tabular embedding via attention"""
    def __init__(self, dim=768, num_heads=12, num_layers=4):
        super().__init__()

        # Multi-head attention layers
        self.fusion_layers = nn.ModuleList([
            nn.MultiheadAttention(
                embed_dim=dim,
                num_heads=num_heads,
                batch_first=True,
                dropout=0.1
            )
            for _ in range(num_layers)
        ])

        # Layer norms
        self.norms = nn.ModuleList([nn.LayerNorm(dim) for _ in range(num_layers)])

        # Feed-forward networks
        self.ffns = nn.ModuleList([
            nn.Sequential(
                nn.Linear(dim, 4 * dim),

```



```

        nn.GELU(),
        nn.Dropout(0.1),
        nn.Linear(4 * dim, dim)
    )
    for _ in range(num_layers)
])

def forward(self, patch_tokens, tabular_embed):
    """
    Args:
        patch_tokens: (B, 196, 768) from DINO
        tabular_embed: (B, 768) from TabularEmbedder
    Returns:
        fused: (B, 768)
    """
    # Expand tabular to sequence format
    tab_seq = tabular_embed.unsqueeze(1)  # (B, 1, 768)

    # Apply fusion layers
    for attn, norm, ffn in zip(self.fusion_layers, self.norms, self.ffns):
        # Cross-attention: patches query tabular
        attn_out, _ = attn(query=patch_tokens, key=tab_seq, value=tab_seq)
        patch_tokens = norm(patch_tokens + attn_out)

        # Feed-forward
        patch_tokens = norm(patch_tokens + ffn(patch_tokens))

    # Global average pooling + residual
    fused = patch_tokens.mean(dim=1) + tabular_embed  # (B, 768)
    return fused

# Unit test
if __name__ == '__main__':
    fusion = CrossModalFusion()
    patches = torch.randn(2, 196, 768)
    tab_emb = torch.randn(2, 768)
    output = fusion(patches, tab_emb)
    assert output.shape == (2, 768)
    print("✓ CrossModalFusion test passed")

```

**Deliverable:** CrossModalFusion class in model.py

## PERSON 4: Prediction Heads & Complete Model

**Time:** 3 hours

```

# Task 4B: Multi-task heads + model integration
import torch
import torch.nn as nn

class BiomassPredictionHeads(nn.Module):
    """Multi-task heads with shared bottleneck"""
    def __init__(self, in_dim=768, hidden_dim=512):
        super().__init__()

```

```

# Shared bottleneck (exploits task correlation)
self.shared = nn.Sequential(
    nn.Linear(in_dim, hidden_dim),
    nn.LayerNorm(hidden_dim),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(hidden_dim, hidden_dim),
    nn.LayerNorm(hidden_dim),
    nn.ReLU()
)

# Task-specific heads
self.heads = nn.ModuleDict({
    'Dry_Green_g': self._make_head(hidden_dim),
    'Dry_Dead_g': self._make_head(hidden_dim),
    'Dry_Clover_g': self._make_head(hidden_dim),
    'GDM_g': self._make_head(hidden_dim),
    'Dry_Total_g': self._make_head(hidden_dim),
})

def _make_head(self, in_dim):
    return nn.Sequential(
        nn.Linear(in_dim, 256), nn.ReLU(), nn.Dropout(0.2),
        nn.Linear(256, 128), nn.ReLU(),
        nn.Linear(128, 1),
        nn.Softplus(beta=1.0) # Ensures positive outputs
    )

def forward(self, x):
    shared = self.shared(x)
    return {target: head(shared).squeeze(-1) for target, head in self.heads.items()}

# Complete model
class MultimodalBiomassModel(nn.Module):
    """Complete architecture: DINO + Tabular + Fusion + Heads"""
    def __init__(self, num_states, num_species):
        super().__init__()
        self.dino_backbone = DINOBackbone(freeze_backbone_pct=0.7)
        self.tabular_embedder = TabularEmbedder(num_states, num_species)
        self.fusion = CrossModalFusion(dim=768, num_heads=12, num_layers=4)
        self.heads = BiomassPredictionHeads(in_dim=768, hidden_dim=512)

    def forward(self, images, tabular_dict):
        patch_tokens = self.dino_backbone(images)
        tabular_embed = self.tabular_embedder(tabular_dict)
        fused = self.fusion(patch_tokens, tabular_embed)
        predictions = self.heads(fused)
        return predictions

# Full integration test
if __name__ == '__main__':
    model = MultimodalBiomassModel(num_states=20, num_species=35)

    # Dummy batch
    images = torch.randn(2, 3, 518, 518)

```

```

tabular = {
    'ndvi': torch.randn(2), 'height': torch.randn(2),
    'date_sin': torch.randn(2), 'date_cos': torch.randn(2),
    'date_sin2': torch.randn(2), 'date_cos2': torch.randn(2),
    'state_id': torch.randint(0, 20, (2,)), 'species_id': torch.randint(0, 35, (2,))
}

preds = model(images, tabular)
assert len(preds) == 5
assert all(preds[t].shape == (2,) for t in preds)

print("✓ Full model test passed")
print(f" Predictions: {list(preds.keys())}")
print(f" Sample: {preds['Dry_Total_g']}")

```

**Deliverable:** BiomassPredictionHeads + MultimodalBiomassModel in model.py (complete file)

## PHASE 3: TRAINING PIPELINE

**Timeline:** Days 5-7 (3 days)

**All 4 people contribute**

### PERSON 1: Loss Function

**Time:** 2 hours

```

# Task 1C: Weighted R² loss with constraints
import torch

def weighted_r2_loss_with_constraints(preds, targets, weights, lambda_constraint=0.05):
    """
    Competition loss function:
    1. Weighted R² (matches evaluation metric)
    2. Soft constraint penalty (biological validity)
    """
    weighted_loss = 0.0
    metrics = {}

    # Per-target R² loss
    for target, weight in weights.items():
        y_true = targets[target]
        y_pred = preds[target]

        # R² = 1 - (SS_res / SS_tot)
        ss_res = torch.sum((y_true - y_pred) ** 2)
        ss_tot = torch.sum((y_true - y_true.mean()) ** 2)
        r2 = 1.0 - (ss_res / (ss_tot + 1e-8))

        # Loss = 1 - R² (minimize to maximize R²)
        loss_component = weight * (1.0 - r2)
        weighted_loss += loss_component

    metrics[f'r2_{target}'] = r2.detach().item()

```

```

# Constraint penalty: Component sum ≤ Total
component_sum = (preds['Dry_Green_g'] +
                 preds['Dry_Dead_g'] +
                 preds['Dry_Clover_g'])
violation = torch.relu(component_sum - preds['Dry_Total_g'] * 1.05)
constraint_penalty = torch.mean(violation ** 2)

total_loss = weighted_loss + lambda_constraint * constraint_penalty

metrics['constraint_violation'] = constraint_penalty.detach().item()
metrics['weighted_loss'] = weighted_loss.detach().item()

return total_loss, metrics

# Test
if __name__ == '__main__':
    preds = {
        'Dry_Green_g': torch.tensor([50.0, 60.0]),
        'Dry_Dead_g': torch.tensor([20.0, 25.0]),
        'Dry_Clover_g': torch.tensor([10.0, 15.0]),
        'GDM_g': torch.tensor([60.0, 75.0]),
        'Dry_Total_g': torch.tensor([80.0, 100.0]),
    }
    targets = {k: v + torch.randn_like(v) * 5 for k, v in preds.items()}
    weights = {'Dry_Green_g': 0.1, 'Dry_Dead_g': 0.1, 'Dry_Clover_g': 0.1,
              'GDM_g': 0.2, 'Dry_Total_g': 0.5}

    loss, metrics = weighted_r2_loss_with_constraints(preds, targets, weights)
    print(f"✓ Loss test passed: {loss.item():.4f}")
    print(f" R² Dry_Total: {metrics['r2_Dry_Total_g']:.4f}")

```

**Deliverable:** loss.py

## PERSON 2: Training Loop

**Time:** 3 hours

```

# Task 2C: Training & validation functions
import torch
from tqdm import tqdm

def train_one_epoch(model, dataloader, optimizer, scheduler, device, loss_fn, weights):
    """Train for one epoch"""
    model.train()
    total_loss = 0.0
    metrics_sum = {}

    pbar = tqdm(dataloader, desc='Training')
    for images, tabular, targets in pbar:
        # Move to device
        images = images.to(device)
        tabular = {k: v.to(device) for k, v in tabular.items()}
        targets = {k: v.to(device) for k, v in targets.items()}

```

```

    # Forward
    preds = model(images, tabular)
    loss, metrics = loss_fn(preds, targets, weights)

    # Backward
    optimizer.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    optimizer.step()

    # Accumulate
    total_loss += loss.item()
    for k, v in metrics.items():
        metrics_sum[k] = metrics_sum.get(k, 0) + v

    # Update progress
    pbar.set_postfix({
        'loss': f"{loss.item():.4f}",
        'r2_Total': f"{metrics.get('r2_Dry_Total_g', 0):.4f}"
    })

scheduler.step()

avg_loss = total_loss / len(dataloader)
avg_metrics = {k: v / len(dataloader) for k, v in metrics_sum.items()}

return avg_loss, avg_metrics

@torch.no_grad()
def validate(model, dataloader, device, weights):
    """Validate model"""
    model.eval()
    all_preds = {t: [] for t in weights.keys()}
    all_targets = {t: [] for t in weights.keys()}

    for images, tabular, targets in tqdm(dataloader, desc='Validating'):
        images = images.to(device)
        tabular = {k: v.to(device) for k, v in tabular.items()}

        preds = model(images, tabular)

        for target in weights.keys():
            all_preds[target].append(preds[target].cpu().numpy())
            all_targets[target].append(targets[target].cpu().numpy())

    # Concatenate
    all_preds = {k: np.concatenate(v) for k, v in all_preds.items()}
    all_targets = {k: np.concatenate(v) for k, v in all_targets.items()}

    # Compute R2
    weighted_r2 = 0.0
    print("\nPer-Target R2:")
    for target, weight in weights.items():
        y_true = all_targets[target]
        y_pred = all_preds[target]

```

```

ss_res = np.sum((y_true - y_pred) ** 2)
ss_tot = np.sum((y_true - y_true.mean()) ** 2)
r2 = 1.0 - (ss_res / (ss_tot + 1e-8))

weighted_r2 += weight * r2
print(f" {target:20s}: {r2:.4f} (w={weight})")

print(f"\n Weighted R²: {weighted_r2:.4f}")
return weighted_r2

```

**Deliverable:** train\_utils.py (training & validation functions)

## PERSON 3: Main Training Script

**Time:** 4 hours

```

# Task 3C: Complete training script with 5-fold CV
import torch
from torch.utils.data import DataLoader
from torch.optim import AdamW
from torch.optim.lr_scheduler import CosineAnnealingWarmRestarts
from sklearn.model_selection import StratifiedKFold
import pandas as pd
import numpy as np
from dataset import BiomassDataset
from transforms import get_train_transforms, get_val_transforms
from model import MultimodalBiomassModel
from loss import weighted_r2_loss_with_constraints
from train_utils import train_one_epoch, validate

def main():
    # Config
    IMG_SIZE = 518
    BATCH_SIZE = 32
    EPOCHS = 120
    DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'

    # Competition weights
    WEIGHTS = {
        'Dry_Green_g': 0.1,
        'Dry_Dead_g': 0.1,
        'Dry_Clover_g': 0.1,
        'GDM_g': 0.2,
        'Dry_Total_g': 0.5,
    }

    # Load data
    train_df = pd.read_csv('data/train_processed.csv')

    # Stratification: state + date quantile
    train_df['strata'] = (
        train_df['State'].astype(str) + '_' +
        pd.qcut(train_df['Sampling_Date'], q=5, labels=False).astype(str)

```

```

)

# 5-Fold CV
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

for fold, (train_idx, val_idx) in enumerate(skf.split(train_df, train_df['strata'])):
    print(f"\n{'='*60}")
    print(f"FOLD {fold}")
    print(f"{'='*60}")

    # Split
    train_fold = train_df.iloc[train_idx]
    val_fold = train_df.iloc[val_idx]

    # Datasets
    train_dataset = BiomassDataset(
        train_fold, 'data/train_images',
        get_train_transforms(IMG_SIZE), mode='train'
    )
    val_dataset = BiomassDataset(
        val_fold, 'data/train_images',
        get_val_transforms(IMG_SIZE), mode='train'
    )

    # Dataloaders
    train_loader = DataLoader(
        train_dataset, batch_size=BATCH_SIZE,
        shuffle=True, num_workers=4, pin_memory=True
    )
    val_loader = DataLoader(
        val_dataset, batch_size=BATCH_SIZE,
        shuffle=False, num_workers=4, pin_memory=True
    )

    # Model
    model = MultimodalBiomassModel(
        num_states=20, # Adjust based on your data
        num_species=35 # Adjust based on your data
    ).to(DEVICE)

    # Optimizer with differential LR
    optimizer = AdamW([
        {'params': model.dino_backbone.parameters(), 'lr': 5e-5},
        {'params': model.tabular_embedder.parameters(), 'lr': 1e-4},
        {'params': model.fusion.parameters(), 'lr': 1e-4},
        {'params': model.heads.parameters(), 'lr': 1e-4},
    ], weight_decay=1e-5)

    # Scheduler
    scheduler = CosineAnnealingWarmRestarts(optimizer, T_0=10, T_mult=2, eta_min=1e-6)

    # Training loop
    best_r2 = -np.inf
    patience_counter = 0
    patience = 20

```

```

for epoch in range(EPOCHS):
    print(f"\nEpoch {epoch+1}/{EPOCHS}")

    # Train
    train_loss, train_metrics = train_one_epoch(
        model, train_loader, optimizer, scheduler,
        DEVICE, weighted_r2_loss_with_constraints, WEIGHTS
    )

    # Validate
    val_r2 = validate(model, val_loader, DEVICE, WEIGHTS)

    print(f"Train Loss: {train_loss:.4f} | Val R²: {val_r2:.4f}")

    # Save best
    if val_r2 > best_r2:
        best_r2 = val_r2
        torch.save(model.state_dict(), f'models/best_model_fold{fold}.pt')
        patience_counter = 0
        print(f"✓ New best: {best_r2:.4f}")
    else:
        patience_counter += 1

    # Early stopping
    if patience_counter >= patience:
        print(f"Early stopping at epoch {epoch+1}")
        break

    print(f"\nFold {fold} Best R²: {best_r2:.4f}")

if __name__ == '__main__':
    main()

```

**Deliverable:** train.py (complete 5-fold training script)

## PERSON 4: Training Monitoring & Logging

**Time:** 2 hours

```

# Task 4C: Add logging & monitoring utilities
import json
import matplotlib.pyplot as plt
from datetime import datetime

class TrainingLogger:
    """Log training metrics to JSON and visualize"""
    def __init__(self, log_dir='logs'):
        self.log_dir = log_dir
        self.logs = {'folds': {}}
        os.makedirs(log_dir, exist_ok=True)

    def log_epoch(self, fold, epoch, train_loss, train_metrics, val_r2):
        """Log one epoch"""
        if fold not in self.logs['folds']:

```



```

        self.logs['folds'][fold] = []

    self.logs['folds'][fold].append({
        'epoch': epoch,
        'train_loss': train_loss,
        'train_metrics': train_metrics,
        'val_r2': val_r2,
        'timestamp': datetime.now().isoformat()
    })

def save(self):
    """Save logs to JSON"""
    with open(f'{self.log_dir}/training_log.json', 'w') as f:
        json.dump(self.logs, f, indent=2)

def plot_fold(self, fold):
    """Plot training curves for one fold"""
    data = self.logs['folds'][fold]
    epochs = [d['epoch'] for d in data]
    train_loss = [d['train_loss'] for d in data]
    val_r2 = [d['val_r2'] for d in data]

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

    ax1.plot(epochs, train_loss, label='Train Loss')
    ax1.set_xlabel('Epoch')
    ax1.set_ylabel('Loss')
    ax1.set_title(f'Fold {fold} - Training Loss')
    ax1.legend()
    ax1.grid(True)

    ax2.plot(epochs, val_r2, label='Val R²', color='green')
    ax2.axhline(y=0.85, color='r', linestyle='--', label='Target')
    ax2.set_xlabel('Epoch')
    ax2.set_ylabel('R²')
    ax2.set_title(f'Fold {fold} - Validation R²')
    ax2.legend()
    ax2.grid(True)

    plt.tight_layout()
    plt.savefig(f'{self.log_dir}/fold{fold}_curves.png')
    plt.close()

# Usage: Add to train.py
# logger = TrainingLogger()
# logger.log_epoch(fold, epoch, train_loss, train_metrics, val_r2)
# logger.save()
# logger.plot_fold(fold)

```

**Deliverable:** logger.py + integration into train.py

## PHASE 4: INFERENCE & SUBMISSION

**Timeline:** Days 8-9 (2 days)

**All 4 people contribute**

### PERSON 1: TTA Inference

**Time:** 3 hours

```
# Task 1D: Test-Time Augmentation inference
import torch
import numpy as np
from PIL import Image
from transforms import get_tta_transforms

@torch.no_grad()
def predict_with_tta(model, image_path, tabular_dict, tta_transforms, device):
    """
    Predict single image with 8-view TTA
    Returns: dict of averaged predictions
    """
    model.eval()

    # Load image
    image = np.array(Image.open(image_path).convert('RGB'))

    # Store predictions
    all_preds = {t: [] for t in ['Dry_Green_g', 'Dry_Dead_g', 'Dry_Clover_g', 'GDM_g', 'I

    # Apply each TTA view
    for transform in tta_transforms:
        aug_image = transform(image=image)['image'].unsqueeze(0).to(device)
        tab_batch = {k: v.unsqueeze(0).to(device) for k, v in tabular_dict.items()}

        preds = model(aug_image, tab_batch)

        for target in all_preds.keys():
            all_preds[target].append(preds[target].cpu().numpy()[0])

    # Average across TTA views
    return {t: np.mean(all_preds[t]) for t in all_preds.keys()}

# Test
if __name__ == '__main__':
    from model import MultimodalBiomassModel

    model = MultimodalBiomassModel(20, 35)
    model.load_state_dict(torch.load('models/best_model_fold0.pt'))
    model.eval()

    tabular = {
        'ndvi': torch.tensor(0.5), 'height': torch.tensor(0.3),
        'date_sin': torch.tensor(0.7), 'date_cos': torch.tensor(0.3),
        'date_sin2': torch.tensor(0.1), 'date_cos2': torch.tensor(0.9),
        'state_id': torch.tensor(5), 'species_id': torch.tensor(10)
```

```

}

tta_transforms = get_tta_transforms()
preds = predict_with_tta(model, 'data/train_images/ID0001.jpg', tabular, tta_transforms)

print("✓ TTA inference test passed")
print(f" Predictions: {preds}")

```

**Deliverable:** tta\_inference.py

## PERSON 2: Constraint Enforcement

**Time:** 2 hours

```

# Task 2D: Post-processing constraint enforcement
def enforce_constraints(predictions):
    """
    5-step biological constraint enforcement
    Ensures physically valid predictions
    """

    # Step 1: Non-negativity
    total = max(0, predictions['Dry_Total_g'])
    green = max(0, predictions['Dry_Green_g'])
    dead = max(0, predictions['Dry_Dead_g'])
    clover = max(0, predictions['Dry_Clover_g'])

    # Step 2: Component sum ≤ Total (with 5% tolerance)
    component_sum = green + dead + clover
    if component_sum > total * 1.05:
        scale = (total * 1.05) / (component_sum + 1e-6)
        green *= scale
        dead *= scale
        clover *= scale

    # Step 3: Clover ratio limit (biological)
    if clover > total * 0.35:
        clover = total * 0.35

    # Step 4: Recalculate GDM
    gdm = green + clover
    gdm = min(gdm, total)

    # Step 5: Recalculate Dead
    dead = total - gdm
    dead = max(0, dead)

    # Final validation
    gdm = min(gdm, total)

    return {
        'Dry_Total_g': total,
        'Dry_Green_g': green,
        'Dry_Dead_g': dead,
        'Dry_Clover_g': clover,
    }

```

```

        'GDM_g': gdm
    }

# Test
if __name__ == '__main__':
    # Test case: violates constraints
    preds = {
        'Dry_Green_g': 50.0,
        'Dry_Dead_g': 30.0,
        'Dry_Clover_g': 25.0, # Sum = 105
        'GDM_g': 75.0,
        'Dry_Total_g': 95.0 # Total < sum!
    }

    corrected = enforce_constraints(preds)

    print("Original predictions:")
    print(f"  Green + Dead + Clover = {preds['Dry_Green_g'] + preds['Dry_Dead_g'] + preds['Dry_Clover_g']}")
    print(f"  Total = {preds['Dry_Total_g']}")

    print("\nCorrected predictions:")
    print(f"  Green + Dead + Clover = {corrected['Dry_Green_g'] + corrected['Dry_Dead_g'] + corrected['Dry_Clover_g']}")
    print(f"  Total = {corrected['Dry_Total_g']:.2f}")
    print(f"  Constraint satisfied: {corrected['Dry_Green_g'] + corrected['Dry_Dead_g'] + corrected['Dry_Clover_g']} < {corrected['Dry_Total_g']}")

```

**Deliverable:** postprocess.py

## PERSON 3: Ensemble Inference Pipeline

**Time:** 4 hours

```

# Task 3D: Complete ensemble inference (5 folds × 8 TTA)
import torch
import pandas as pd
import numpy as np
from tqdm import tqdm
from model import MultimodalBiomassModel
from tta_inference import predict_with_tta
from postprocess import enforce_constraints
from transforms import get_tta_transforms

def ensemble_inference(test_df, img_dir, num_folds=5, device='cuda'):
    """
    Complete ensemble pipeline:
    - Load 5 fold models
    - Apply 8-view TTA per fold
    - Total: 40 forward passes per image
    - Enforce constraints
    """
    # Load models
    models = []
    for fold in range(num_folds):
        model = MultimodalBiomassModel(num_states=20, num_species=35)
        model.load_state_dict(torch.load(f'models/best_model_fold{fold}.pt'))

```

```

        model.to(device).eval()
        models.append(model)
        print(f"✓ Loaded fold {fold} model")

# TTA transforms
tta_transforms = get_tta_transforms(518)

# Inference
all_predictions = []

for idx in tqdm(range(len(test_df)), desc='Ensemble Inference'):
    img_id = test_df.loc[idx, 'sample_id']
    img_path = f"{img_dir}/{img_id}.jpg"

    # Extract tabular features
    tabular = {
        'ndvi': torch.tensor(test_df.loc[idx, 'ndvi_norm'], dtype=torch.float32),
        'height': torch.tensor(test_df.loc[idx, 'height_norm'], dtype=torch.float32),
        'date_sin': torch.tensor(test_df.loc[idx, 'date_sin'], dtype=torch.float32),
        'date_cos': torch.tensor(test_df.loc[idx, 'date_cos'], dtype=torch.float32),
        'date_sin2': torch.tensor(test_df.loc[idx, 'date_sin2'], dtype=torch.float32),
        'date_cos2': torch.tensor(test_df.loc[idx, 'date_cos2'], dtype=torch.float32),
        'state_id': torch.tensor(test_df.loc[idx, 'state_id'], dtype=torch.long),
        'species_id': torch.tensor(test_df.loc[idx, 'species_id'], dtype=torch.long),
    }

    # Collect predictions from all folds
    fold_predictions = []

    for model in models:
        preds = predict_with_tta(model, img_path, tabular, tta_transforms, device)
        fold_predictions.append(preds)

    # Average across folds
    final_preds = {
        target: np.mean([p[target] for p in fold_predictions])
        for target in ['Dry_Green_g', 'Dry_Dead_g', 'Dry_Clover_g', 'GDM_g', 'Dry_Tot']
    }

    # Enforce constraints
    final_preds = enforce_constraints(final_preds)

    # Store
    all_predictions.append({
        'sample_id': img_id,
        **final_preds
    })

return all_predictions

# Usage
if __name__ == '__main__':
    import pickle

    # Load test data
    test_df = pd.read_csv('data/test.csv')

```

```

# Apply same preprocessing
with open('data/preprocessor.pkl', 'rb') as f:
    preprocessor = pickle.load(f)
test_processed = preprocessor.transform(test_df)

# Run ensemble inference
predictions = ensemble_inference(test_processed, 'data/test_images')

print(f"\n✓ Inference complete: {len(predictions)} images")

```

**Deliverable:** inference.py

## PERSON 4: Submission Generation & Validation

**Time:** 2 hours

```

# Task 4D: Generate submission file
import pandas as pd

def generate_submission(predictions, output_path='submission.csv'):
    """
    Convert predictions to competition format:
    sample_id format: {imageid}__{target}
    """
    submission_rows = []

    for pred in predictions:
        img_id = pred['sample_id']
        for target in ['Dry_Green_g', 'Dry_Dead_g', 'Dry_Clover_g', 'GDM_g', 'Dry_Total_g']:
            submission_rows.append({
                'sample_id': f"{img_id}__{target}",
                'target': pred[target]
            })

    submission_df = pd.DataFrame(submission_rows)
    submission_df.to_csv(output_path, index=False)

    print(f"\n✓ Submission saved: {output_path}")
    print(f"  Rows: {len(submission_df)}")
    print(f"  Expected: {len(predictions) * 5}")

    # Validation checks
    print("\nValidation:")
    print(f"  ✓ No NaN values: {submission_df['target'].isna().sum() == 0}")
    print(f"  ✓ All positive: {(submission_df['target'] >= 0).all()}")
    print(f"  ✓ Reasonable range: {submission_df['target'].min():.2f} - {submission_df['target'].max():.2f}")

    # Sample preview
    print("\nSample rows:")
    print(submission_df.head(10))

    return submission_df

```

```

# Usage
if __name__ == '__main__':
    # Load predictions from inference
    predictions = pd.read_pickle('predictions.pkl') # Saved by Person 3

    # Generate submission
    submission = generate_submission(predictions)

    # Final check: constraint satisfaction
    print("\nConstraint Check (sample 10 images):")
    for i in range(10):
        img_preds = predictions[i]
        component_sum = (img_preds['Dry_Green_g'] +
                        img_preds['Dry_Dead_g'] +
                        img_preds['Dry_Clover_g'])
        valid = component_sum <= img_preds['Dry_Total_g'] * 1.05
        print(f" Image {i}: {'✓' if valid else '✗'} (sum={component_sum:.2f}, total={in

```

**Deliverable:** submission.py + final submission.csv

## COORDINATION & INTEGRATION

### Daily Sync Meetings (15 min each)

- **Day 1:** Phase 1 kickoff, assign tasks
- **Day 2:** Phase 1 review, Phase 2 kickoff
- **Day 3:** Phase 2 review, test full model
- **Day 4:** Phase 3 kickoff, start training fold 0
- **Day 5:** Monitor training, adjust hyperparameters if needed
- **Day 6:** Continue training folds 1-4
- **Day 7:** Phase 4 kickoff, test inference
- **Day 8:** Run full ensemble inference
- **Day 9:** Final submission review & submit

### Critical Integration Points

1. **After Phase 1:** All 4 test dataloader together
2. **After Phase 2:** All 4 test full model forward pass
3. **After Phase 3:** All 4 review training logs
4. **After Phase 4:** All 4 validate submission file

## Final Deliverables Checklist

- ☐ data/train\_processed.csv (Person 2)
- ☐ dataset.py (Person 3)
- ☐ transforms.py (Person 4)
- ☐ model.py (All 4 contributed modules)
- ☐ loss.py (Person 1)
- ☐ train\_utils.py (Person 2)
- ☐ train.py (Person 3)
- ☐ logger.py (Person 4)
- ☐ models/best\_model\_fold{0-4}.pt (Person 3 trains)
- ☐ tta\_inference.py (Person 1)
- ☐ postprocess.py (Person 2)
- ☐ inference.py (Person 3)
- ☐ submission.py (Person 4)
- ☐ submission.csv (Final output)

**EVERYONE LEARNS THE FULL PIPELINE. EVERYONE CONTRIBUTES TO EVERY PHASE. LET'S WIN THIS! 🏆**